Hàm trong JavaScript

Function declaration, function expression, arrow function.

Mục tiêu bài học

- Định nghĩa hàm bằng function declaration, function expression và arrow function
- Sử dụng tham số, giá trị mặc định và giá trị trả về
- Hiểu hàm là giá trị có thể truyền như biến

1. Khai báo hàm

Hàm là khối lệnh có tên để tái sử dụng. Function declaration được nâng lên đầu phạm vi (hoisting) nên có thể gọi trước khi khai báo.

```
function cong(a, b) {
  return a + b;
}
console.log(cong(2, 3)); // 5
```

2. Function expression và arrow function

Function expression gán hàm cho một biến. Arrow function là cú pháp ngắn gọn: `const add = (a, b) => a + b;`. Với một tham số có thể bỏ ngoặc: `x => x * 2`. Arrow function không có `this` riêng, nó kế thừa `this` từ ngữ cảnh bao quanh.

```
const nhan = function (a, b) { return a * b; };
const binhPhuong = (x) => x * x;
```

3. Tham số mặc định và rest

Tham số có thể có giá trị mặc định: `function chao(ten = "bạn") {...}`. Cú pháp rest `...args` gom các đối số còn lại thành một mảng.

```
function tong(...so) {
  return so.reduce((t, x) => t + x, 0);
}
console.log(tong(1, 2, 3, 4)); // 10
```

4. Hàm là giá trị

Hàm có thể được gán vào biến, truyền làm đối số cho hàm khác (callback) hoặc được trả về từ hàm. Đây là nền tảng của xử lý sự kiện và lập trình bất đồng bộ.

Tóm tắt

- Ba cách viết hàm: declaration, expression, arrow.
- Tham số mặc định và rest giúp hàm linh hoạt hơn.
- Hàm là giá trị nên có thể làm callback.

Bài tập vận dụng

1. Viết hàm kiểm tra một số có phải số chẵn hay không, bằng cả function thường và arrow function.
2. Viết hàm `tong(...so)` tính tổng số lượng đối số bất kỳ.
3. Viết hàm nhận một mảng và một callback rồi áp dụng callback cho từng phần tử.